

Identify Overfitting from Training Logs and Solve It

Overfitting - When a model performs very well on training data but poorly on unseen data.

Identify Overfitting from Training Logs and Solve It

What is the Problem?

Observation	Meaning
Training Accuracy ↑	Model learns training data very well
Validation Accuracy Stagnates or ↓	Model performs poorly on unseen data
Result	Overfitting

Overfitting Example

Epoch	Training Accuracy	Validation Accuracy
1	80%	78%
5	92%	86%
10	98%	87%
15	99%	86%

Conclusion: Training accuracy keeps increasing, but validation accuracy stops improving ⇒ **Overfitting**.

Libraries Used

Code	Purpose
<code>Sequential</code>	Creates neural network model
<code>Dense</code>	Fully connected layer
<code>Flatten</code>	Converts 2D image into 1D vector
<code>Dropout</code>	Prevents overfitting
<code>EarlyStopping</code>	Stops training when validation performance stops improving
<code>mnist</code>	Handwritten digit dataset
<code>to_categorical</code>	One-hot encoding of labels

Load MNIST Dataset

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

Data set	Images
----------	--------

Training	60,000
----------	--------

Testing 10,000

MNIST Details

Property	Value
----------	-------

Classes	10 Digits (0–9)
---------	-----------------

Image Size	28 × 28
------------	---------

Channels	Grayscale (1 Channel)
----------	-----------------------

Normalize Data

$x_{\text{train}}, x_{\text{test}} = x_{\text{train}} / 255.0, x_{\text{test}} / 255.0$

Purpose

Before	After
--------	-------

0 - 255	0 - 1
---------	-------

Benefits:

- Faster training
 - Better convergence
-

Convert Labels

```
y_train, y_test = to_categorical(y_train), to_categorical(y_test)
```

Example

Original Label	One-Hot Encoding
3	<code>[0,0,0,1,0,0,0,0,0,0]</code>
7	<code>[0,0,0,0,0,0,0,0,1,0]</code>

Model Architecture

Layer	Purpose	Output
<code>Flatten(input_shape=(28,28))</code>	Convert image to vector	784
<code>Dense(256, activation='relu')</code>	Hidden Layer 1	256 Neurons
<code>Dropout(0.3)</code>	Prevent Overfitting	Drops 30% neurons
<code>Dense(128, activation='relu')</code>	Hidden Layer 2	128 Neurons
<code>Dropout(0.3)</code>	Prevent Overfitting	Drops 30% neurons

```
Dense(10,  
activation='softmax')
```

Output Layer

10 Classes

What is Dropout?

Dropout(0.3)

Value

Meaning

0.3 Randomly disables 30%
neurons during training

Example

Total Neurons	Active	Disabled
100	70	30

Benefit

**Without
Dropout**

With Dropout

Overfitting likely

Overfitting
reduced

Memorizes
training data

Learns
generalized
patterns

Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Parameter	Purpose
Adam	Optimizer
categorical_crossentropy	Multi-class loss function
accuracy	Performance metric

Early Stopping

```
early_stop = EarlyStopping(  
    patience=3,  
    restore_best_weights=True  
)
```

Meaning

Parameter	Purpose
<code>patience=3</code>	Wait 3 epochs for improvement
<code>restore_best_weights=True</code>	Restore best model weights

Example

Epo ch	Validation Accuracy
-------------------	--------------------------------

5	95%
---	-----

6	95%
---	-----

7	95%
---	-----

8	95%
---	-----

After 3 epochs without improvement, training stops automatically.

Training

```
model.fit(  
    x_train,  
    y_train,  
    validation_data=(x_test, y_test),  
    epochs=30,  
    callbacks=[early_stop]  
)
```

Parameters

Parameter	Purpose
<code>x_train</code>	Training images
<code>y_train</code>	Training labels

<code>validation_data</code>	Validation dataset
<code>epochs=30</code>	Maximum training cycles
<code>callbacks=[early_stop]</code>	Enable early stopping

How Overfitting is Solved

Technique	Purpose
Dropout	Randomly disables neurons to prevent memorization
Early Stopping	Stops training before overfitting starts
Validation Data	Monitors model performance on unseen data

Interview Questions

Question	Answer
What is Overfitting?	When a model performs well on training data but poorly on unseen data.

How do you identify overfitting?	Training accuracy increases while validation accuracy stagnates or decreases.
What is Dropout?	A regularization technique that randomly disables neurons during training.
What is Early Stopping?	Stops training when validation performance stops improving.
Why use <code>restore_best_weights=True</code> ?	Restores the model weights from the best validation performance epoch.
Why use MNIST?	It is a simple handwritten digit dataset used for learning deep learning concepts.

One-Line Summary

This code detects and reduces overfitting in an MNIST classification model using Dropout layers and EarlyStopping, improving the model's ability to generalize to unseen data.

table format

Question 4: Overfitting and Solution (Table Format)

Code	Purpose	Explanation
------	---------	-------------

<code>from tensorflow.keras.models import Sequential</code>	Import Model	Used to create a sequential neural network.
<code>from tensorflow.keras.layers import Dense, Flatten, Dropout</code>	Import Layers	Used to build neural network layers.
<code>from tensorflow.keras.callbacks import EarlyStopping</code>	Import Callback	Stops training automatically when validation performance stops improving.
<code>from tensorflow.keras.datasets import mnist</code>	Load Dataset	Imports MNIST handwritten digit dataset.
<code>from tensorflow.keras.utils import to_categorical</code>	One-Hot Encoding	Converts labels into categorical format.
<code>mnist.load_data()</code>	Load Data	Loads training and testing datasets.
<code>x_train / 255.0</code>	Normalize Data	Converts pixel values from 0-255 to 0-1.

<code>to_categorical(y_train)</code>	Convert Labels	Converts labels into one-hot encoded vectors.
<code>Sequential([...])</code>	Create Model	Defines neural network architecture.
<code>Flatten(input_shape=(28,28))</code>	Flatten Layer	Converts 28×28 image into 784 features.
<code>Dense(256, activation='relu')</code>	Hidden Layer 1	Learns complex patterns from data.
<code>Dropout(0.3)</code>	Regularization	Randomly disables 30% neurons to reduce overfitting.
<code>Dense(128, activation='relu')</code>	Hidden Layer 2	Learns deeper features.
<code>Dropout(0.3)</code>	Regularization	Further reduces overfitting.
<code>Dense(10, activation='softmax')</code>	Output Layer	Predicts one of 10 digit classes (0-9).
<code>model.compile()</code>	Configure Model	Sets optimizer, loss function, and metric.

<code>optimizer='adam'</code>	Optimizer	Updates weights efficiently.
<code>loss='categorical_crossentropy'</code>	Loss Function	Measures prediction error for multi-class classification.
<code>metrics=['accuracy']</code>	Evaluation Metric	Tracks model accuracy.
<code>EarlyStopping(patience=3)</code>	Early Stopping	Stops training if validation performance doesn't improve for 3 epochs.
<code>restore_best_weights=True</code>	Best Model Recovery	Restores weights from best-performing epoch.
<code>model.fit()</code>	Train Model	Starts model training.
<code>validation_data=(x_test, y_test)</code>	Validation Data	Monitors performance on unseen data.
<code>epochs=30</code>	Maximum Epochs	Allows training up to 30 epochs.

<code>callbacks=[early_stop]</code>	Apply Early Stopping	Enables automatic stopping.
-------------------------------------	----------------------	-----------------------------

Model Architecture Summary

Layer	Input Shape	Output Shape	Purpose
Input	28 × 28	28 × 28	MNIST Image
Flatten	28 × 28	784	Convert image to vector
Dense	784	256	Feature Learning
Drop out	256	256	Reduce Overfitting
Dense	256	128	Deep Feature Learning
Drop out	128	128	Reduce Overfitting
Dense	128	10	Classification Output

Overfitting Identification

Training Accuracy	Validation Accuracy	Conclusion
Increasing	Increasing	Good Learning
Increasing	Stagnating	Overfitting Starts
Increasing	Decreasing	Severe Overfitting

Example:

Epo ch	Training Accuracy	Validation Accuracy
1	80%	79%
5	92%	90%
10	98%	91%
15	99%	90%

Result: Training accuracy improves but validation accuracy stops improving ⇒ **Overfitting**.

Solution Summary

Technique

Purpose

<code>Dropout(0.3)</code>	Randomly removes neurons during training to prevent memorization.
<code>EarlyStopping</code>	Stops training before overfitting occurs.
<code>Validation Data</code>	Monitors model performance on unseen data.
<code>restore_best_weights=True</code>	Uses the best-performing model instead of the last epoch.

Interview Questions

Question	Answer
What is Overfitting?	When a model performs very well on training data but poorly on unseen data.
How do you identify Overfitting?	Training accuracy increases while validation accuracy stagnates or decreases.
What is Dropout?	A regularization technique that randomly disables neurons during training.
What is Early Stopping?	Stops training when validation performance stops improving.
Why use <code>patience=3</code> ?	Waits for 3 epochs before stopping training.

Why use
`restore_best_weights=True`?

Restores the model weights from the best validation epoch.

=====